TUGAS PERKULIAHAN
BASIS DATA

MODEL DATA RELASIONAL DAN QUERYNYA

Dosen Pengajar:
Khabib Mustofa



disusun oleh
Kelompok Basis Data (Identitas Dummy)
4 anggota; lihat daftar pada Bab II

PROGRAM STUDI SARJANA ILMU KOMPUTER
DEPARTEMEN ILMU KOMPUTER DAN ELEKTRONIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS GADJAH MADA
YOGYAKARTA

2026

DAFTAR ISI

Halaman Judul **i**

I Petunjuk dan Bentuk Penugasan **1**

1.1 Petunjuk Tugas 1

1.2 Detail Isi Penugasan 2

1.2.1 Domain Kasus 2

1.2.2 Output/Outcome Penugasan 3

1.3 Contoh Soal dan Pengerjaannya 4

II Hasil dan Pembahasan **6**

2.1 Entity Relationship Diagram 6

2.2 Skema Tabel-Tabel dalam Basis Data 7

2.2.1 DDL SQL 8

2.3 Pertanyaan Bisnis dan Penyelesaian dengan SQL 13

2.3.1 Soal Nomor 1 13

2.3.2 Soal Nomor 2 14

2.3.3 Soal Nomor 3 14

2.3.4 Soal Nomor 4 16

2.3.5 Soal Nomor 5 16

2.4 Validasi 18

2.5 Kesimpulan 18

DAFTAR TABEL

DAFTAR GAMBAR

2.1 ERD basis data akademik 7

BAB I

Petunjuk dan Bentuk Penugasan

Integritas Akademik

Seluruh karya yang dikumpulkan harus merupakan hasil pemikiran dan kerja mahasiswa sendiri. Menyalin kode atau laporan dari sumber lain tanpa atribusi yang jelas adalah pelanggaran integritas akademik dan akan dikenai tindakan sesuai peraturan universitas.

1.1 Petunjuk Tugas

Tugas yang diberikan adalah menyangkut Model Relasional dan Query atas Model Relasional:

- Aljabar Relasional
- *Structured Query Language (SQL)*

Jawaban tugas dikemas dalam satu bab (bab 2), yang bagian-bagiannya telah disediakan dalam template ini. Isi Bab tersebut dapat dikembangkan sesuai kebutuhan, namun tidak untuk dikurangi.

Deliverables

Tugas terdiri atas empat bagian

1. menerjemahkan batasan dan proses bisnis ke dalam diagram ER (*Entity Relationship Diagram*). Usulan disertai penjelasan proses pembentukan ERD
2. merancang dan mengusulkan skema basis data relasional yang sesuai dengan ERD yang diusulkan
3. Jika diberikan query, mengusulkan representasi query dalam ekspresi aljabar relasional
4. Mengusulkan penyelesaian pertanyaan dalam bentuk query SQL. Bila ada lebih dari satu kemungkinan query (dengan operasi atau klausa yang berbeda), query alternatif juga perlu dituliskan

Ketentuan Tugas

1. Pekerjaan dapat dikerjakan secara berkelompok dengan anggota kelompok maksimal 4 orang.
2. Setiap anggota kelompok **harus** berkontribusi secara memadai, dan menguasai materi yang dihasilkan oleh kelompoknya.
3. Pekerjaan harus disajikan secara *jelas, lengkap dan rapi, termasuk penggunaan bahasa formal yang mudah dipahami*
4. Pekerjaan harus dari pemikiran anggota tim, dan disepakati oleh semua anggota

1.2 Detail Isi Penugasan

1.2.1 Domain Kasus

Diinginkan untuk mengelola data akademik suatu perguruan tinggi dengan batasan sebagai berikut:

Batasan Proses

- Setiap kegiatan akademik perlu dicatat dengan transparan dan mempertahankan kronologi/historis
- Desain penyimpanan data mengakomodasi hal-hal dan proses berikut:
  - Seorang mahasiswa diperkenankan mengulang matakuliah tanpa batas berapa kali
  - Nilai matakuliah (apabila pernah mengulang) yang digunakan sebagai nilai untuk transkrip adalah **nilai pengambilan terakhir**
  - Matakuliah ada yang bersifat WAJIB dan ada PILIHAN.
  - Matakuliah dapat diprasyarati matakuliah lainnya
  - Seorang dosen dapat mengajar suatu matakuliah di suatu semester lebih di satu kelas (paralel)
  - Beberapa matakuliah dapat ditawarkan di semster genap ataupun semester ganjil
  - Suatu kelas untuk suatu matakuliah diajar penuh oleh seorang dosen (tidak secara team teching)
  - Nilai matakuliah dan bobot yang diberikan adalah adalah

Nilai	Bobot
A	4,00
A-	3,75
A/B	3,50
B+	3,25
B	3,00
B-	2,75

Nilai	Bobot
B/C	2,50
C+	2,25
C	2
C/D	2,5
D	1,00
D	0,00

- Rumus untuk menghitung IPK dari n mata kuliah adalah:

$$IPK = \frac{\sum_{i=1}^{n} \text{sks}_i.\text{bobot}_i}{\sum_{i=1}^{n} \text{sks}_i}$$

1.2.2 Output/Outcome Penugasan

Berikut adalah hal-hal minimal yang harus menjadi output/outcome penugasan.

Deliverables

1. E-R diagram (ERD) yang sesuai hal-hal di atas! (dimulai dari identifikasi: entitas, relasi, kardinalitas, dsb.)
2. Skema lengkap tabel-tabel basis data yang sesuai ERD yang dibuat; misal dalam bentuk
   `Mhs(nim, nama, jenisKelamin, prodi,...)`
   `Dosen(nip, ...)`
   Dan tulis juga **perintah DDL SQL untuk definisi tabel-tabel** di atas (lengkap dengan semua *constraint*)!
3. Perintah DML untuk pertanyaan-pertanyaan di bawah ini:
   (a) Daftar nama mata kuliah pilihan dan mata kuliah yang memprasyaratinya (cukup info nama dan kode matakuliah saja). Matakuliah yang tidak memiliki prasyarat tetap ditampilkan!
   (b) Daftar Matakuliah pilihan yang diambil oleh mahasiswa Ilmu Komputer angkatan '2024' pada semester ganjil tahun akademik 2026/2027!
   (c) Daftar Matakuliah pilihan yang favorit untuk mahasiswa Ilmu Komputer, yaitu tiga matakuliah dengan pengambil terbanyak dalam tiga tahun terakhir!
   (d) Daftar mahasiswa yang memiliki Indeks Prestasi Semester ganjil tahun 2025/2026 tidak kurang dari 3,25! (pahami perbedaan antara IP Semester dan IP Kumulatif)

(e) Daftar mahasiswa dan IP Kumulatif yang diperolehnya saat ini! (matakuliah yang pernah diulang cukup dihitung sekali)

4. Query dalam bentuk ekspresi aljabar relasional untuk setiap pertanyaan/query yang diberikan!

Untuk setiap query di atas, langkah pengerjaan perlu diuraikan:

1. identifikasi tabel-tabel yang perlu dilibatkan
2. penentuan kriteria join dan jenis join: INNER JOIN, NATURAL JOIN, LEFT/RIGHT JOIN atau OUTER JOIN
3. Perintah SQL yang valid. Apabila memerlukan VIEW sebagai temporary table, silahkan juga dituliskan secara runtut.
4. apabila ada lebih dari satu SQL query yang *equivalent*, akan menjadi nilai tambah apabila Anda bisa menuliskan serta query tersebut!

1.3 Contoh Soal dan Pengerjaannya

Misal diberikan query:

> *Tampilkan daftar mata kuliah pilihan yang belum pernah diambil/diikuti oleh mahasiswa*

maka contoh jawaban adalah sebagai berikut:

Query nomor 1

1. **Tabel yang diperlukan dalam query:** MATAKULIAH (MK), PENGAMBILANKULIAH (PK)
2. bila dengan operasi SET DIFFERENCE tidak ada proses join, sehingga tidak ada kriteria join. Sebagai alternatif, dapat digunakan LEFT JOIN, dengan kriteria `Mk.idMatakuliah= Pk.idMatakuliah`
3. **Query yang diperlukan**

```
SELECT * FROM Matakuliah Mk WHERE (sifat='P') and
    idMatakuliah NOT IN
     (SELECT idMatakuliah FROM PengambilanKuliah Pk)
```

Listing I.1: pendekatan SET DIFFERENCE

4. Alternatif SQL yang ekuivalen

```
SELECT Mk.* FROM Matakuliah Mk LEFT JOIN
    PesertaKuliah Pk ON (Mk.idMatakuliah=Pk.
    idMatakuliah)
     WHERE Mk.sifat='P' and Pk.idMatakuliah is NULL
```

Listing I.2: dengan LEFT JOIN

```
SELECT * FROM Matakuliah Mk WHERE (sifat='P') and
     NOT EXISTS
     (SELECT idMatakuliah FROM PengambilanKuliah Pk
         WHERE Pk.idMatakuliah=Mk.idMatakuliah)
```

Listing I.3: dengan EXISTS

5. Ekspresi Aljabar Relasional

$$(\pi_{\text{idMatakuliah}}(\sigma_{\text{sifat='P'}}(\text{Matakuliah})) - \pi_{\text{idMatakuliah}}(\text{Matakuliah})) \bowtie \text{Matakuliah}$$

$$DIFF1 \leftarrow \pi_{\text{idMatakuliah}}(\sigma_{\text{sifat='P'}}(\text{Matakuliah}))$$
$$DIFF2 \leftarrow (DIFF1 - \pi_{\text{idMatakuliah}}(\text{Matakuliah}))$$
$$DIFF2 \bowtie \text{Matakuliah}$$

BAB II

Hasil dan Pembahasan

title=Daftar Anggota Kelompok

1. Anggota 1, NIM: 00/000001/PA/00001 — analisis proses bisnis dan ERD.
2. Anggota 2, NIM: 00/000002/PA/00002 — skema relasional dan DDL.
3. Anggota 3, NIM: 00/000003/PA/00003 — query 1–3 dan aljabar relasional.
4. Anggota 4, NIM: 00/000004/PA/00004 — query 4–5, integrasi, dan validasi.

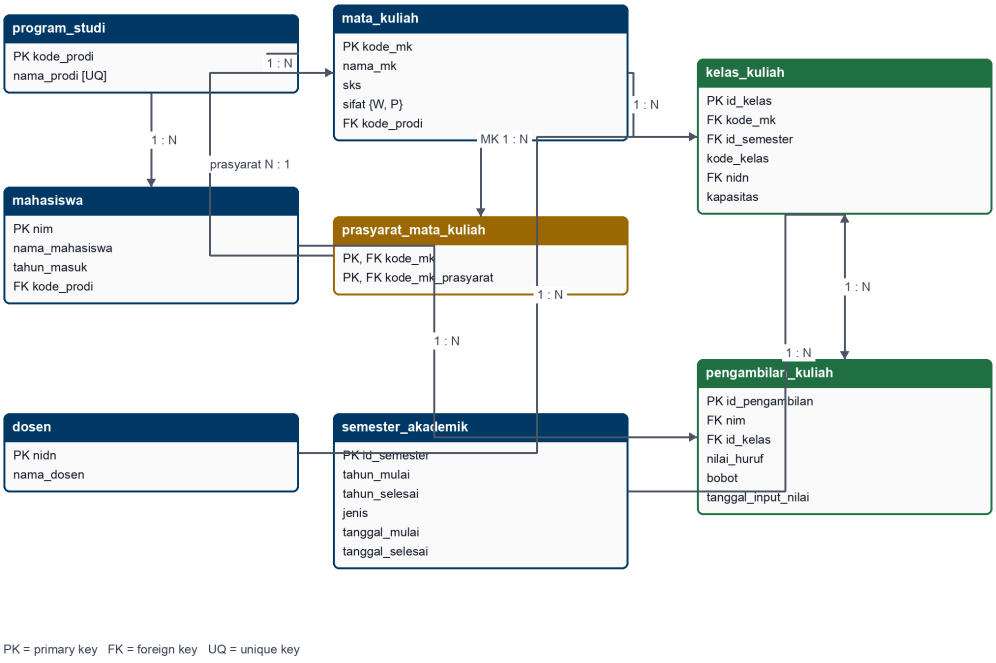
Catatan: identitas di atas bersifat sementara dan harus diganti sebelum pengumpulan.

2.1 Entity Relationship Diagram

Basis data dirancang untuk mempertahankan kronologi kegiatan akademik. Delapan entitas yang digunakan ialah `program_studi`, `mahasiswa`, `dosen`, `mata_kuliah`, `prasyarat_mata_kuliah`, `semester_akademik`, `kelas_kuliah`, dan `pengambilan_kuliah`. Pemisahan data semester, kelas, dan pengambilan membuat setiap riwayat kuliah tersimpan sebagai baris tersendiri sehingga pengulangan mata kuliah tidak menghapus nilai terdahulu.

Satu program studi memiliki banyak mahasiswa dan mata kuliah. Satu mata kuliah dapat ditawarkan dalam banyak kelas pada semester yang berbeda, sedangkan satu kelas hanya mengacu kepada satu mata kuliah, satu semester, dan satu dosen. Dengan demikian, dosen dapat mengajar lebih dari satu kelas paralel, tetapi satu kelas tidak menerapkan *team teaching*. Relasi banyak-ke-banyak antara mahasiswa dan kelas direalisasikan oleh `pengambilan_kuliah`.

Prasyarat dimodelkan sebagai relasi rekursif pada `mata_kuliah` melalui tabel penghubung `prasyarat_mata_kuliah`. Desain ini memungkinkan satu mata kuliah memiliki beberapa prasyarat dan satu mata kuliah menjadi prasyarat bagi beberapa mata kuliah lain. Atribut `sifat` membedakan mata kuliah wajib (W) dan pilihan (P).



Gambar 2.1: ERD basis data akademik

Nilai mengikuti bobot A=4,00; A-=3,75; A/B=3,50; B+=3,25; B=3,00; B-=2,75; B/C=2,50; C+=2,25; C=2,00; C/D=1,50; D=1,00; dan E=0,00. Angka C/D dan simbol E tersebut digunakan untuk memperbaiki kekeliruan tipografis pada petunjuk yang menuliskan C/D=2,5 dan D dua kali.

2.2 Skema Tabel-Tabel dalam Basis Data

title=Skema Relasional

- `program_studi(kode_prodi, nama_prodi)`
- `mahasiswa(nim, nama_mahasiswa, tahun_masuk, kode_prodi[FK])`
- `dosen(nidn, nama_dosen)`
- `mata_kuliah(kode_mk, nama_mk, sks, sifat, kode_prodi[FK])`
- `prasyarat_mata_kuliah(kode_mk[FK], kode_mk_prasyarat[FK])`
- `semester_akademik(id_semester, tahun_mulai, tahun_selesai, jenis, tanggal_mulai, tanggal_selesai)`
- `kelas_kuliah(id_kelas, kode_mk[FK], id_semester[FK], kode_kelas, nidn[FK], kapasitas)`

- pengambilan_kuliah(id_pengambilan, nim[FK], id_kelas[FK], nilai_huruf, bobot, tanggal_input_nilai)

Setiap tabel merepresentasikan satu jenis fakta sehingga ketergantungan atribut nonkunci diarahkan kepada kunci tabelnya. Relasi banyak-ke-banyak dipisahkan menjadi tabel penghubung dan atribut program studi, mata kuliah, dosen, serta semester tidak diduplikasi pada riwayat pengambilan. Oleh karena itu, skema memenuhi bentuk normal ketiga (3NF) untuk kebutuhan yang diberikan.

2.2.1 DDL SQL

DDL lengkap beserta data uji tersedia pada berkas `schema.sql`. Definisi inti berikut menunjukkan seluruh tabel dan constraint yang digunakan.

```
DROP DATABASE IF EXISTS tugas_akademik;
CREATE DATABASE tugas_akademik CHARACTER SET utf8mb4 COLLATE
    utf8mb4_unicode_ci;
USE tugas_akademik;

CREATE TABLE program_studi (
    kode_prodi VARCHAR(10) PRIMARY KEY,
    nama_prodi VARCHAR(100) NOT NULL UNIQUE
);

CREATE TABLE mahasiswa (
    nim VARCHAR(20) PRIMARY KEY,
    nama_mahasiswa VARCHAR(100) NOT NULL,
    tahun_masuk YEAR NOT NULL,
    kode_prodi VARCHAR(10) NOT NULL,
    CONSTRAINT fk_mahasiswa_prodi FOREIGN KEY (kode_prodi)
        REFERENCES program_studi(kode_prodi)
        ON UPDATE CASCADE ON DELETE RESTRICT
);

CREATE TABLE dosen (
    nidn VARCHAR(20) PRIMARY KEY,
    nama_dosen VARCHAR(100) NOT NULL
);

CREATE TABLE mata_kuliah (
    kode_mk VARCHAR(10) PRIMARY KEY,
```

```
    nama_mk VARCHAR(100) NOT NULL,
    sks TINYINT UNSIGNED NOT NULL,
    sifat ENUM('W', 'P') NOT NULL,
    kode_prodi VARCHAR(10) NOT NULL,
    CONSTRAINT chk_mk_sks CHECK (sks BETWEEN 1 AND 6),
    CONSTRAINT fk_mk_prodi FOREIGN KEY (kode_prodi)
        REFERENCES program_studi(kode_prodi)
        ON UPDATE CASCADE ON DELETE RESTRICT
);

CREATE TABLE prasyarat_mata_kuliah (
    kode_mk VARCHAR(10) NOT NULL,
    kode_mk_prasyarat VARCHAR(10) NOT NULL,
    PRIMARY KEY (kode_mk, kode_mk_prasyarat),
    CONSTRAINT chk_prasyarat_bukan_diri CHECK (kode_mk <>
       kode_mk_prasyarat),
    CONSTRAINT fk_prasyarat_mk FOREIGN KEY (kode_mk)
        REFERENCES mata_kuliah(kode_mk)
        ON UPDATE RESTRICT ON DELETE CASCADE,
    CONSTRAINT fk_prasyarat_induk FOREIGN KEY (kode_mk_prasyarat
       )
        REFERENCES mata_kuliah(kode_mk)
        ON UPDATE RESTRICT ON DELETE RESTRICT
);

CREATE TABLE semester_akademik (
    id_semester INT AUTO_INCREMENT PRIMARY KEY,
    tahun_mulai YEAR NOT NULL,
    tahun_selesai YEAR NOT NULL,
    jenis ENUM('Ganjil', 'Genap') NOT NULL,
    tanggal_mulai DATE NOT NULL,
    tanggal_selesai DATE NOT NULL,
    CONSTRAINT uq_semester UNIQUE (tahun_mulai, tahun_selesai,
       jenis),
    CONSTRAINT chk_tahun_akademik CHECK (tahun_selesai =
       tahun_mulai + 1),
    CONSTRAINT chk_tanggal_semester CHECK (tanggal_selesai >
       tanggal_mulai)
);

CREATE TABLE kelas_kuliah (
    id_kelas BIGINT AUTO_INCREMENT PRIMARY KEY,
```

```
    kode_mk VARCHAR(10) NOT NULL,
    id_semester INT NOT NULL,
    kode_kelas VARCHAR(10) NOT NULL,
    nidn VARCHAR(20) NOT NULL,
    kapasitas SMALLINT UNSIGNED NOT NULL DEFAULT 40,
    CONSTRAINT uq_kelas UNIQUE (kode_mk, id_semester, kode_kelas
        ),
    CONSTRAINT chk_kapasitas CHECK (kapasitas > 0),
    CONSTRAINT fk_kelas_mk FOREIGN KEY (kode_mk)
        REFERENCES mata_kuliah(kode_mk)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_kelas_semester FOREIGN KEY (id_semester)
        REFERENCES semester_akademik(id_semester)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_kelas_dosen FOREIGN KEY (nidn)
        REFERENCES dosen(nidn)
        ON UPDATE CASCADE ON DELETE RESTRICT
);

CREATE TABLE pengambilan_kuliah (
    id_pengambilan BIGINT AUTO_INCREMENT PRIMARY KEY,
    nim VARCHAR(20) NOT NULL,
    id_kelas BIGINT NOT NULL,
    nilai_huruf ENUM('A', 'A-', 'A/B', 'B+', 'B', 'B-', 'B/C', '
        C+', 'C', 'C/D', 'D', 'E') NULL,
    bobot DECIMAL(3,2) NULL,
    tanggal_input_nilai DATE NULL,
    CONSTRAINT uq_pengambilan UNIQUE (nim, id_kelas),
    CONSTRAINT chk_nilai_bobot CHECK (
        (nilai_huruf IS NULL AND bobot IS NULL) OR
        (nilai_huruf = 'A'   AND bobot = 4.00) OR
        (nilai_huruf = 'A-'  AND bobot = 3.75) OR
        (nilai_huruf = 'A/B' AND bobot = 3.50) OR
        (nilai_huruf = 'B+'  AND bobot = 3.25) OR
        (nilai_huruf = 'B'   AND bobot = 3.00) OR
        (nilai_huruf = 'B-'  AND bobot = 2.75) OR
        (nilai_huruf = 'B/C' AND bobot = 2.50) OR
        (nilai_huruf = 'C+'  AND bobot = 2.25) OR
        (nilai_huruf = 'C'   AND bobot = 2.00) OR
        (nilai_huruf = 'C/D' AND bobot = 1.50) OR
        (nilai_huruf = 'D'   AND bobot = 1.00) OR
        (nilai_huruf = 'E'   AND bobot = 0.00)
```

```
    ),
    CONSTRAINT fk_pengambilan_mahasiswa FOREIGN KEY (nim)
        REFERENCES mahasiswa(nim)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_pengambilan_kelas FOREIGN KEY (id_kelas)
        REFERENCES kelas_kuliah(id_kelas)
        ON UPDATE CASCADE ON DELETE RESTRICT
);

INSERT INTO program_studi VALUES
('IK', 'Ilmu Komputer'),
('EL', 'Elektronika dan Instrumentasi');

INSERT INTO mahasiswa VALUES
('24IK001', 'Aditya Pratama', 2024, 'IK'),
('24IK002', 'Bella Maharani', 2024, 'IK'),
('23IK001', 'Citra Lestari', 2023, 'IK'),
('22IK001', 'Damar Nugraha', 2022, 'IK'),
('24EL001', 'Eka Saputra', 2024, 'EL');

INSERT INTO dosen VALUES
('D001', 'Dr. Andi Wijaya'),
('D002', 'Dr. Bunga Kartika'),
('D003', 'Dr. Candra Kusuma');

INSERT INTO mata_kuliah VALUES
('IK101', 'Dasar Pemrograman', 3, 'W', 'IK'),
('IK201', 'Basis Data', 3, 'W', 'IK'),
('IK301', 'Kecerdasan Buatan', 3, 'P', 'IK'),
('IK302', 'Data Mining', 3, 'P', 'IK'),
('IK303', 'Komputasi Awan', 3, 'P', 'IK'),
('IK304', 'Interaksi Manusia dan Komputer', 3, 'P', 'IK'),
('EL101', 'Rangkaian Elektronika', 3, 'W', 'EL');

INSERT INTO prasyarat_mata_kuliah VALUES
('IK301', 'IK101'),
('IK302', 'IK201'),
('IK303', 'IK201');

INSERT INTO semester_akademik
(id_semester, tahun_mulai, tahun_selesai, jenis, tanggal_mulai,
   tanggal_selesai) VALUES
```

```
(1, 2023, 2024, 'Ganjil', '2023-08-01', '2023-12-31'),
(2, 2024, 2025, 'Ganjil', '2024-08-01', '2024-12-31'),
(3, 2025, 2026, 'Ganjil', '2025-08-01', '2025-12-31'),
(4, 2025, 2026, 'Genap',  '2026-02-01', '2026-06-30'),
(5, 2026, 2027, 'Ganjil', '2026-08-01', '2026-12-31');

INSERT INTO kelas_kuliah
(id_kelas, kode_mk, id_semester, kode_kelas, nidn, kapasitas)
   VALUES
(1,  'IK101', 1, 'A', 'D001', 40),
(2,  'IK201', 2, 'A', 'D002', 40),
(3,  'IK301', 2, 'A', 'D003', 40),
(4,  'IK302', 3, 'A', 'D002', 40),
(5,  'IK303', 3, 'A', 'D001', 40),
(6,  'IK304', 3, 'A', 'D003', 40),
(7,  'IK201', 3, 'A', 'D002', 40),
(8,  'IK301', 5, 'A', 'D003', 40),
(9,  'IK301', 5, 'B', 'D001', 40),
(10, 'IK302', 5, 'A', 'D002', 40),
(11, 'IK304', 5, 'A', 'D003', 40),
(12, 'EL101', 5, 'A', 'D001', 40),
(13, 'IK302', 4, 'A', 'D002', 40);

INSERT INTO pengambilan_kuliah
(nim, id_kelas, nilai_huruf, bobot, tanggal_input_nilai) VALUES
('22IK001', 1,  'B',   3.00, '2024-01-05'),
('22IK001', 3,  'A-',  3.75, '2025-01-05'),
('23IK001', 2,  'B+',  3.25, '2025-01-05'),
('23IK001', 3,  'B',   3.00, '2025-01-05'),
('23IK001', 4,  'A',   4.00, '2026-01-05'),
('23IK001', 5,  'B+',  3.25, '2026-01-05'),
('24IK001', 2,  'C',   2.00, '2025-01-05'),
('24IK001', 7,  'A',   4.00, '2026-01-05'),
('24IK001', 4,  'A-',  3.75, '2026-01-05'),
('24IK001', 5,  'B+',  3.25, '2026-01-05'),
('24IK001', 6,  'A',   4.00, '2026-01-05'),
('24IK002', 7,  'B+',  3.25, '2026-01-05'),
('24IK002', 4,  'B',   3.00, '2026-01-05'),
('24IK002', 6,  'A-',  3.75, '2026-01-05'),
('24IK001', 8,  NULL,  NULL, NULL),
('24IK001', 10, NULL,  NULL, NULL),
('24IK002', 9,  NULL,  NULL, NULL),
```

```
('24IK002', 11, NULL,  NULL, NULL),
('24EL001', 12, NULL,  NULL, NULL),
('22IK001', 13, 'A',   4.00, '2026-07-05');
```

Listing II.1: DDL dan data uji basis data akademik

2.3 Pertanyaan Bisnis dan Penyelesaian dengan SQL

2.3.1 Soal Nomor 1

Daftar nama mata kuliah pilihan dan mata kuliah yang memprasyaratinya. Mata kuliah yang tidak memiliki prasyarat tetap ditampilkan.

1. **Tabel:** `mata_kuliah` dan `prasyarat_mata_kuliah`; `mata_kuliah` dipakai dua kali sebagai mata kuliah utama dan mata kuliah prasyarat.
2. **Join:** `LEFT JOIN` pada `pmk.kode_mk=mk.kode_mk`, kemudian `pre.kode_mk=pmk.kode_mk_p` Outer join diperlukan agar mata kuliah tanpa prasyarat tetap muncul dengan nilai `NULL`.
3. **SQL utama:**

```
SELECT mk.kode_mk, mk.nama_mk,
       pre.kode_mk AS kode_prasyarat,
       pre.nama_mk AS nama_prasyarat
FROM mata_kuliah AS mk
LEFT JOIN prasyarat_mata_kuliah AS pmk
       ON pmk.kode_mk = mk.kode_mk
LEFT JOIN mata_kuliah AS pre
       ON pre.kode_mk = pmk.kode_mk_prasyarat
WHERE mk.sifat = 'P'
ORDER BY mk.kode_mk, pre.kode_mk;
```

4. **Alternatif:** correlated subquery dapat digunakan untuk mengambil prasyarat. Namun, bentuk ini hanya aman jika satu mata kuliah dibatasi memiliki paling banyak satu prasyarat; `LEFT JOIN` lebih umum dan menjadi pilihan utama.
5. **Aljabar relasional:**

$$\pi_{mk.kode_mk,mk.nama_mk,pre.kode_mk,pre.nama_mk}\left(\sigma_{mk.sifat='P'}(\rho_{mk}(MataKuliah)) \bowtie_{mk.kode_mk=}\right.$$

2.3.2 Soal Nomor 2

Daftar mata kuliah pilihan yang diambil mahasiswa Ilmu Komputer angkatan 2024 pada semester ganjil tahun akademik 2026/2027.

1. **Tabel:** `pengambilan_kuliah`, `mahasiswa`, `program_studi`, `kelas_kuliah`, `mata_kuliah`, dan `semester_akademik`.
2. **Join:** seluruh tabel dihubungkan melalui foreign key. Semua join bersifat inner join karena hanya pengambilan yang mempunyai mahasiswa, kelas, mata kuliah, dan semester lengkap yang relevan.
3. **SQL utama:**

```sql
SELECT DISTINCT mk.kode_mk, mk.nama_mk
FROM pengambilan_kuliah AS pk
JOIN mahasiswa AS m ON m.nim = pk.nim
JOIN program_studi AS ps ON ps.kode_prodi = m.kode_prodi
JOIN kelas_kuliah AS kk ON kk.id_kelas = pk.id_kelas
JOIN mata_kuliah AS mk ON mk.kode_mk = kk.kode_mk
JOIN semester_akademik AS sa ON sa.id_semester = kk.
   id_semester
WHERE ps.nama_prodi = 'Ilmu Komputer'
  AND m.tahun_masuk = 2024 AND mk.sifat = 'P'
  AND sa.jenis = 'Ganjil'
  AND sa.tahun_mulai = 2026 AND sa.tahun_selesai = 2027
ORDER BY mk.kode_mk;
```

4. **Alternatif:** mata kuliah pilihan dapat dipilih dari `mata_kuliah` dan disaring dengan correlated `EXISTS`. Bentuk lengkap tersedia pada `queries.sql` dan menghasilkan himpunan mata kuliah yang sama.
5. **Aljabar relasional:**

$$\pi_{kode_mk,nama_mk}\sigma_{nama_prodi='Ilmu\ Komputer'\wedge tahun_masuk=2024\wedge sifat='P'\wedge jenis='Ganjil'\wedge tahun_mulai=}$$

2.3.3 Soal Nomor 3

Tiga mata kuliah pilihan favorit bagi mahasiswa Ilmu Komputer berdasarkan jumlah pengambil terbanyak dalam tiga tahun terakhir.

Tiga tahun terakhir dioperasionalkan sebagai semester dengan `tahun_mulai` 2024 sampai 2026. Jumlah dihitung sebagai mahasiswa berbeda menggunakan `COUNT(DISTINCT nim)` agar mahasiswa yang mengulang mata kuliah dalam rentang tersebut tidak menggandakan jumlah peminat.

1. **Tabel:** `mata_kuliah`, `kelas_kuliah`, `semester_akademik`, `pengambilan_kuliah`, `mahasiswa`, dan `program_studi`.
2. **Join:** inner join berdasarkan foreign key. Hasil dikelompokkan per mata kuliah.
3. **SQL utama:**

```sql
WITH jumlah_pengambil AS (
  SELECT mk.kode_mk, mk.nama_mk,
         COUNT(DISTINCT pk.nim) AS jumlah_mahasiswa
  FROM mata_kuliah mk
  JOIN kelas_kuliah kk ON kk.kode_mk = mk.kode_mk
  JOIN semester_akademik sa ON sa.id_semester = kk.
     id_semester
  JOIN pengambilan_kuliah pk ON pk.id_kelas = kk.id_kelas
  JOIN mahasiswa m ON m.nim = pk.nim
  JOIN program_studi ps ON ps.kode_prodi = m.kode_prodi
  WHERE mk.sifat='P' AND ps.nama_prodi='Ilmu Komputer'
    AND sa.tahun_mulai BETWEEN 2024 AND 2026
  GROUP BY mk.kode_mk, mk.nama_mk
), peringkat AS (
  SELECT jumlah_pengambil.*,
         DENSE_RANK() OVER (ORDER BY jumlah_mahasiswa DESC)
             AS peringkat
  FROM jumlah_pengambil
)
SELECT * FROM peringkat WHERE peringkat <= 3
ORDER BY peringkat, kode_mk;
```

4. **Alternatif:** `ORDER BY jumlah_mahasiswa DESC LIMIT 3` menghasilkan tepat tiga baris, tetapi dapat menghilangkan mata kuliah yang seri pada batas peringkat. Karena itu, `DENSE_RANK()` dipilih sebagai jawaban utama.
5. **Aljabar relasional diperluas:**

$$R \leftarrow \sigma_{sifat='P' \wedge nama_prodi='Ilmu\ Komputer' \wedge 2024 \le tahun_mulai \le 2026}(MK \bowtie KK \bowtie SA \bowtie PK \bowtie$$

$$J \leftarrow \gamma_{kode_mk,nama_mk;\ count(distinct\ nim) \to jumlah}(R), \qquad \tau_{jumlah\ desc}(J)[peringkat \le 3]$$

2.3.4 Soal Nomor 4

Daftar mahasiswa dengan Indeks Prestasi Semester ganjil 2025/2026 tidak kurang dari 3,25.

IPS hanya menggunakan nilai dalam satu semester. Nilai setiap mata kuliah dibobotkan dengan SKS, lalu total mutu dibagi total SKS.

1. **Tabel:** `mahasiswa`, `pengambilan_kuliah`, `kelas_kuliah`, `mata_kuliah`, dan `semester_akademik`.
2. **Join:** inner join berdasarkan NIM, ID kelas, kode mata kuliah, dan ID semester.
3. **SQL utama:**

```
SELECT m.nim, m.nama_mahasiswa,
       ROUND(SUM(mk.sks * pk.bobot) / SUM(mk.sks), 2) AS
          ips
FROM mahasiswa m
JOIN pengambilan_kuliah pk ON pk.nim = m.nim
JOIN kelas_kuliah kk ON kk.id_kelas = pk.id_kelas
JOIN mata_kuliah mk ON mk.kode_mk = kk.kode_mk
JOIN semester_akademik sa ON sa.id_semester = kk.
   id_semester
WHERE sa.jenis='Ganjil' AND sa.tahun_mulai=2025
  AND sa.tahun_selesai=2026 AND pk.bobot IS NOT NULL
GROUP BY m.nim, m.nama_mahasiswa
HAVING SUM(mk.sks * pk.bobot) / SUM(mk.sks) >= 3.25
ORDER BY m.nim;
```

4. **Alternatif:** agregasi IPS ditempatkan dalam CTE `nilai_ips`, kemudian disaring dengan `WHERE ips >= 3.25`. Hasilnya sama dengan penggunaan `HAVING`.
5. **Aljabar relasional diperluas:**

$$R \leftarrow \sigma_{jenis='Ganjil' \wedge tahun_mulai=2025 \wedge tahun_selesai=2026 \wedge bobot \neq null}(M \bowtie PK \bowtie KK \bowtie MK \bowtie S$$

$$I \leftarrow \gamma_{nim,nama;\ sum(sks \cdot bobot)/sum(sks) \rightarrow ips}(R), \qquad \sigma_{ips \geq 3.25}(I)$$

2.3.5 Soal Nomor 5

Daftar mahasiswa dan IP Kumulatif saat ini. Mata kuliah yang pernah diulang dihitung satu kali menggunakan nilai pengambilan terakhir.

Urutan pengambilan ditentukan oleh `tanggal_mulai` semester dan `id_pengambilan` sebagai pemecah seri. `ROW_NUMBER()` memberi nomor satu pada pengambilan terbaru untuk setiap pasangan mahasiswa–mata kuliah.

1. **Tabel:** `mahasiswa`, `pengambilan_kuliah`, `kelas_kuliah`, `mata_kuliah`, dan `semester_akademik`.
2. **Join:** inner join berdasarkan foreign key. CTE mempartisi data menurut NIM dan kode mata kuliah.
3. **SQL utama:**

```
WITH riwayat AS (
  SELECT pk.nim, kk.kode_mk, mk.sks, pk.bobot,
         ROW_NUMBER() OVER (
           PARTITION BY pk.nim, kk.kode_mk
           ORDER BY sa.tanggal_mulai DESC, pk.
              id_pengambilan DESC
         ) AS urutan
  FROM pengambilan_kuliah pk
  JOIN kelas_kuliah kk ON kk.id_kelas = pk.id_kelas
  JOIN mata_kuliah mk ON mk.kode_mk = kk.kode_mk
  JOIN semester_akademik sa ON sa.id_semester = kk.
     id_semester
  WHERE pk.bobot IS NOT NULL
)
SELECT m.nim, m.nama_mahasiswa,
       ROUND(SUM(r.sks*r.bobot)/SUM(r.sks),2) AS ipk
FROM mahasiswa m
JOIN riwayat r ON r.nim=m.nim AND r.urutan=1
GROUP BY m.nim, m.nama_mahasiswa
ORDER BY m.nim;
```

4. **Alternatif:** baris terbaru dapat dipilih dengan `NOT EXISTS`, yaitu menerima baris ketika tidak ada pengambilan mata kuliah yang sama pada semester yang lebih baru. Bentuk lengkap tersedia pada `queries.sql` dan telah diuji menghasilkan IPK yang sama.
5. **Aljabar relasional diperluas:**

$$R \leftarrow \sigma_{bobot \neq null}(PK \bowtie KK \bowtie MK \bowtie SA)$$

$$P \leftarrow \rho_{R_1}(R) \times \rho_{R_2}(R)$$
$$U \leftarrow \sigma_{R_1.nim=R_2.nim \,\wedge\, R_1.kode_mk=R_2.kode_mk}(P)$$
$$B \leftarrow \sigma_{R_1.tanggal_mulai<R_2.tanggal_mulai}(U)$$
$$L \leftarrow R - \pi_{R_1.*}(B)$$

$$\gamma_{nim,nama;\ sum(sks\cdot bobot)/sum(sks)\rightarrow ipk}(Mahasiswa \bowtie L)$$

2.4 Validasi

DDL dan seluruh query dijalankan pada MySQL. Data uji mencakup mata kuliah tanpa prasyarat, kelas paralel, mahasiswa dari program studi berbeda, nilai satu semester, dan pengulangan mata kuliah. Query utama dan alternatif pada soal 1, 2, 4, dan 5 menghasilkan relasi yang sama. Pada soal 3, perbedaan yang disengaja ialah `DENSE_RANK()` mempertahankan seluruh mata kuliah yang seri, sedangkan alternatif `LIMIT 3` selalu menghasilkan tepat tiga baris.

2.5 Kesimpulan

Model relasional yang diusulkan dapat menyimpan histori akademik tanpa kehilangan kronologi, merepresentasikan prasyarat rekursif, mendukung kelas paralel, dan memastikan satu dosen bertanggung jawab penuh atas satu kelas. Pemisahan entitas dan penerapan constraint menjaga integritas data. Kombinasi join, agregasi, fungsi jendela, CTE, dan anti-join dapat menjawab seluruh pertanyaan bisnis, termasuk perhitungan IPS dan IPK berdasarkan pengambilan terakhir.